NURU

Final product technical report | Scope, implementation, validation, and limits

Project. Nuru is a source-linked return information service. Public users browse information without accounts; authorized staff review and publish content through a web administration workflow. The repository also contains an Expo mobile prototype.

1. Scope alignment and feature validation

Planned feature	Purpose	Current status	Evidence in repository
Browse/search/filter	Find relevant guidance by keyword, category, or destination.	Implemented	frontend/src/pages/Explore.tsx; backend/information/views.py
Article details and sources	Show full guidance and its cited source/date.	Implemented	frontend/src/pages/InformationDetail.tsx; InformationEntry model/serializer
Public issue reports	Let readers flag possibly stale or incorrect entries.	Implemented	frontend/src/pages/ReportInformation.tsx; backend/reports/
Admin authoring and publishing	Allow authorized staff to maintain public information.	Implemented	JWT login, admin pages, Django permissions
Review, version history, resolution	Make publication evidence and editorial outcomes traceable.	Implemented	SourceReview, EntryRevision, report resolution fields; migrations and tests
Multilingual content	Improve access in local languages.	Partial	rn/rw/sw UI catalogs, reviewed article/guide translation fields and fallback; content coverage/native review not established
Mobile access	Extend core public tasks to phones.	Partial prototype	nuru-mobile/ implements browse, search, reports, bookmarks and guides; device validation/release not documented
Offline/SMS/maps/push	Support low-connectivity channels and discovery.	Not implemented	Identified as future work; no production implementation found

Scope change. The project expanded beyond the web MVP with an experimental mobile client and stronger editorial controls (versioned review, revision snapshots, and report resolution attribution). Offline support and mobile distribution remain future work. The mobile branch is a prototype, not a validated release.

2. Technical product demonstration

Walkthrough sequence. Start at Home, search a keyword, filter by category/destination, open an article and source, change language, submit a report, sign in as staff, edit a draft, record a suitable source review, publish it, then resolve a report with notes. Show rejected publication without a current review and the empty search state. The same API powers web and mobile clients.

Core flows and technical logic

Flow	Implementation and logic
Discovery	URL-backed filters in React call Axios service functions. DRF filters published entries by title, summary, body and reviewed translations; category and commune filters are applied in the API.
Editorial publishing	Staff-only JWT permissions protect create/update/review operations. A suitable review is tied to content_version; substantive edits advance the version, so old approval cannot authorize changed text. Source name is required.
Reports	Anonymous users submit an open report. Staff resolution requires notes; the server records who resolved it and when. Reopening clears active resolution fields.
History	InformationEntry.save() records JSON snapshots of substantive fields and labels, uniquely keyed by entry/version. Admin history views are read-only.
Language and mobile	Reviewed translations are selected when complete; otherwise English fallback is shown. Mobile keeps saved article IDs in AsyncStorage and needs a live API to reopen their content.

Technical architecture

React + TypeScript web client and Expo/React Native prototype connect to an Axios/JSON API, then Django REST Framework serializers, views and permission classes, Django ORM, and PostgreSQL. Local backend tests override the database with in-memory SQLite. Frontend pages/components, service modules, typed domain models, backend apps, and persistence models separate presentation, transport, policy, and data concerns.

Code quality and problem solving

Encapsulation and separation of concerns appear in model, serializer, view, service, component and page boundaries. Reusable typed UI components reduce duplicated rendering. The custom version-aware publication rule, reviewed-translation fallback/search, and versioned revision snapshots address domain-specific data integrity needs. Django models and React components provide reusable objects; inheritance/polymorphism are used mainly through framework base classes and are not forced into the domain model.

3. Testing and validation evidence

Approach 1: automated API and workflow tests

Why: publication, access control, search and report transitions affect what users can trust and who can change data. **How:** Django REST Framework API tests exercise authenticated and anonymous requests; model and Django admin tests check revision snapshots and staff workflows. **Recorded results:** the submitted `python manage.py test --verbosity=2` run discovered 28 tests, applied all migrations to PostgreSQL test database `test_nuru_db`, reported no system-check issues, and finished with 28 passing tests in 549.276 seconds. Django warned that `backend/staticfiles/` was absent; this did not fail the run. The test database already existed, and the pasted output shows it was recreated after confirmation. **Improvements evidenced:** the publication validation checks the version that will be saved, preventing edits from reusing an old review; server-owned review/resolution attribution and public draft filtering are also tested.

Representative input/response cases

Input / condition	Expected system response	Test evidence
Publish with no suitable current-version review	400 validation error; remains unpublished	PublicationWorkflowTests
Public GET for an unpublished entry	Entry omitted from list; detail returns 404	PublicationWorkflowTests
Search case-insensitively across body or reviewed translation	Matching published entry returned	InformationSearchTests
Resolve report with blank notes	400 validation error; status remains open	ReportResolutionTests
Edit source content after review	Version increments; old review no longer authorizes publish	PublicationWorkflowTests / model history tests

Approach 2: visual interface review from captured screens

Why: information access depends on readable screens and clear task steps. **How:** inspect the committed Home, Explore, detail, report, login, dashboard, and authoring screenshots in docs/screenshots. **Observed:** captures show the principal public and admin screens and give a baseline for a walkthrough. The walkthrough plan was updated to call out empty results, validation failures, language fallback, and device/browser identification. The captures do not document screen-reader use, measured usability, viewport sizes, or browser versions. This is visual coverage, not proof of compatibility; add a live cross-device walkthrough and record findings before claiming those results.

Testing status caveat

The supplied Django run provides PostgreSQL-backed test evidence. It does not provide browser matrix results, mobile emulator/physical-device validation, stress/load results, or measured CPU/memory/FPS. The mobile README says mocked API contract tests pass and TypeScript syntax transpilation succeeds, while full dependency/type checking and live device/backend checks remain incomplete.

4. Performance, compatibility, and submission evidence

Performance and compatibility

The responsive web layouts use CSS flex/grid and fluid sizing. This is a design implementation detail, not a measured compatibility result. No browser/OS/version matrix or low-end/high-end hardware metrics were found in the repository. No runtime performance numbers are claimed. For the final video, record device/browser/OS versions, viewport, API response conditions, and any observed failures; use the same scenario on at least one phone-class and one higher-capacity device if available.

Demonstration capture checklist

- Show browser and app versions plus the test device/emulator; identify when the backend is local or deployed.
- Demonstrate public search/filter/article/source/report, then admin review/publish and report resolution.
- Include invalid inputs (missing review, blank resolution notes, no search match) and the visible response.
- Show a second language with a reviewed translation and the English fallback when one is unavailable.
- Capture a mobile browse/search/report flow and state clearly if it was performed on emulator or physical hardware.
- State that performance metrics were not collected unless you measure and report them during the recording.

Repository evidence and remaining submission steps

Repository: https://github.com/abu00123/Nuru
Walkthrough folder currently listed in README:
https://drive.google.com/drive/folders/1zUbNJ20X7dENxw5n-wL5GDB7oSWUOisk?usp=drive_link
Report source evidence: backend/information/tests.py, backend/reports/tests.py, backend/information/models.py, backend/information/views.py, frontend/src/, nuru-mobile/README.md, docs/screenshots/.

Submission checks: ensure the Drive folder contains the final accessible video; upload this PDF and replace the README report URL if a hosted link is required; include the video and, if required, a built APK in the Attempt 2 archive. The repository currently contains source code and a mobile prototype; an APK and a packaged walkthrough video were not identified during this review.

Evidence basis: code and documentation present in the Nuru repository as of 1 November 2026. Status claims are limited to artifacts visible there; missing runtime evidence is marked as such.